

Widzenie komputerowe

Laboratorium mediaPipe

Marcin Mikuła

Kod rozwiązania, prawie taki sam jak podany na zajęciach z drobnymi poprawkami.

```
import cv2
import mediapipe as mp
import numpy as np
import time

mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(min_detection_confidence=0.5,
min_tracking_confidence=0.5)

mp_drawing = mp.solutions.drawing_utils
drawing_spec = mp_drawing.DrawingSpec(thickness=1, circle_radius=1)

cap = cv2.VideoCapture(0)

while cap.isOpened():
    success, image = cap.read()
    if not success:
        print("Ignoring empty frame.")
        continue

    start = time.time()

    # Convert the color space from BGR to RGB for processing
    image_rgb = cv2.cvtColor(cv2.flip(image, 1), cv2.COLOR_BGR2RGB) # Flip
    for selfie-view
    image_rgb.flags.writeable = False

    # Get face mesh results
    results = face_mesh.process(image_rgb)

    # To improve performance
    image_rgb.flags.writeable = True

    # Convert the color space from RGB back to BGR
    image_bgr = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2BGR)

    img_h, img_w, _ = image_bgr.shape
    face_3d = []
    face_2d = []

    if results.multi_face_landmarks:
        for face_landmarks in results.multi_face_landmarks:
            for idx, lm in enumerate(face_landmarks.landmark):
                if idx in [33, 263, 1, 61, 291, 199]: # Select landmarks for
nose, eyes, and mouth
                    if idx == 1:
                        nose_2d = (lm.x * img_w, lm.y * img_h)
                        nose_3d = (lm.x * img_w, lm.y * img_h, lm.z * 3000)
```

```

        x, y = int(lm.x * img_w), int(lm.y * img_h)

        # Get the 2D and 3D coordinates
        face_2d.append([x, y])
        face_3d.append([x, y, lm.z])

    # Convert to NumPy arrays
    face_2d = np.array(face_2d, dtype=np.float64)
    face_3d = np.array(face_3d, dtype=np.float64)

    # Camera matrix
    focal_length = 1 * img_w
    cam_matrix = np.array([[focal_length, 0, img_h / 2],
                           [0, focal_length, img_w / 2],
                           [0, 0, 1]])

    # Distortion parameters
    dist_matrix = np.zeros((4, 1), dtype=np.float64)

    # Solve PnP to get rotation and translation vectors
    success, rot_vec, trans_vec = cv2.solvePnP(face_3d, face_2d,
cam_matrix, dist_matrix)

    # Get rotational matrix and angles
    rmat, _ = cv2.Rodrigues(rot_vec)
    angles, _, _, _, _ = cv2.RQDecomp3x3(rmat)

    x, y, z = angles[0] * 360, angles[1] * 360, angles[2] * 360

    # Determine head direction
    if y < -10:
        text = "Looking Left"
    elif y > 10:
        text = "Looking Right"
    elif x < -10:
        text = "Looking Down"
    elif x > 10:
        text = "Looking Up"
    else:
        text = "Forward"

    # Display direction on the image
    nose_3d_projection, _ = cv2.projectPoints(nose_3d, rot_vec,
trans_vec, cam_matrix, dist_matrix)
    p1 = (int(nose_2d[0]), int(nose_2d[1]))
    p2 = (int(nose_2d[0] + y * 10), int(nose_2d[1] - x * 10))

    cv2.line(image_bgr, p1, p2, (255, 0, 0), 3)
    cv2.putText(image_bgr, text, (20, 50), cv2.FONT_HERSHEY_SIMPLEX,
2, (0, 255, 0), 2)
    cv2.putText(image_bgr, f"x: {x:.2f}", (500, 50),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)

```

```

        cv2.putText(image_bgr, f"y: {y:.2f}", (500, 100),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)
        cv2.putText(image_bgr, f"z: {z:.2f}", (500, 150),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)

    # Draw face mesh
    mp_drawing.draw_landmarks(
        image=image_bgr,
        landmark_list=face_landmarks,
        connections=mp_face_mesh.FACEMESH_CONTOURS,
        landmark_drawing_spec=drawing_spec,
        connection_drawing_spec=drawing_spec
    )

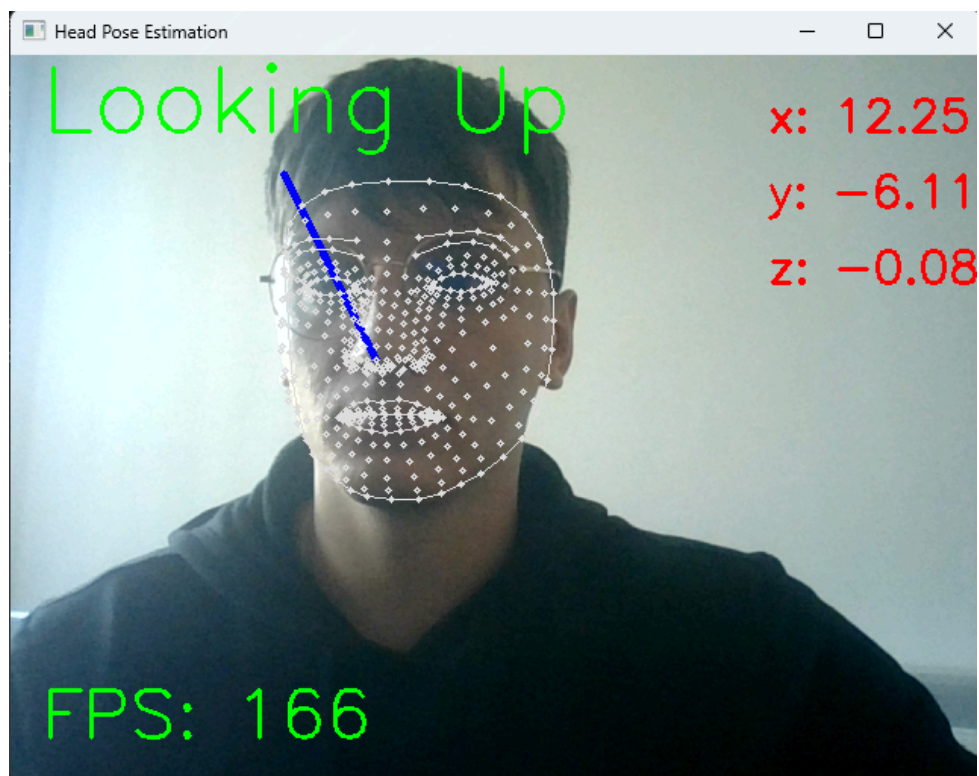
    # Calculate and display FPS
    end = time.time()
    fps = 1 / (end - start)
    cv2.putText(image_bgr, f'FPS: {int(fps)}', (20, 450),
cv2.FONT_HERSHEY_SIMPLEX, 1.5, (0, 255, 0), 2)

    # Show the image
    cv2.imshow('Head Pose Estimation', image_bgr)

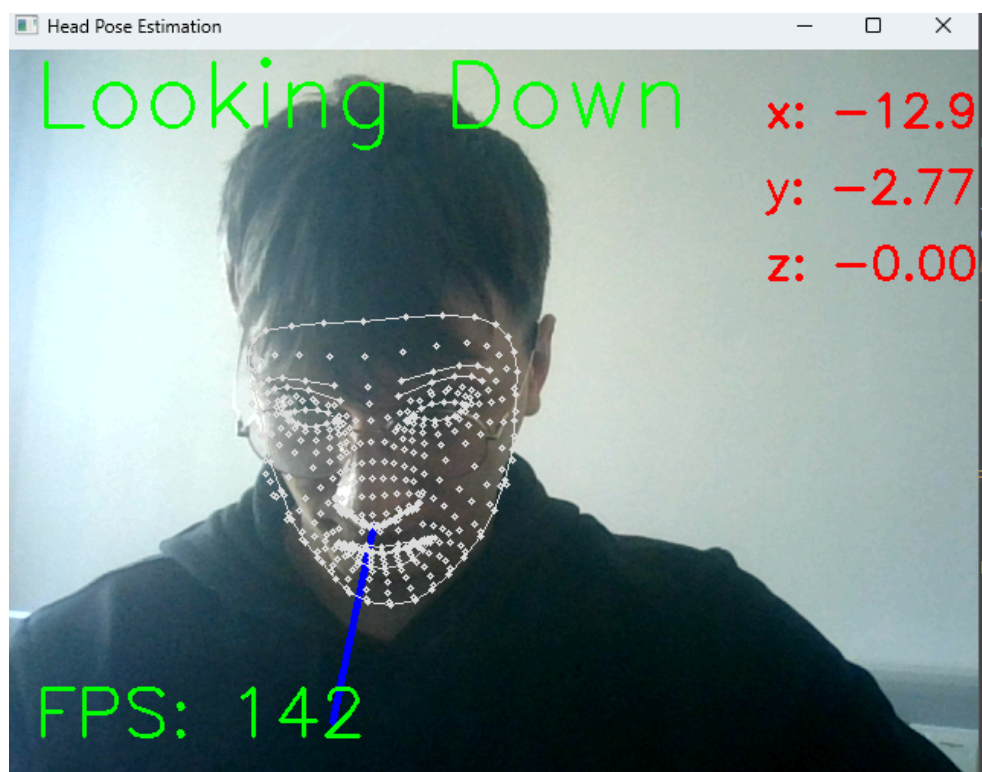
    # Exit on 'ESC' key
    if cv2.waitKey(5) & 0xFF == 27:
        break

cap.release()
cv2.destroyAllWindows()

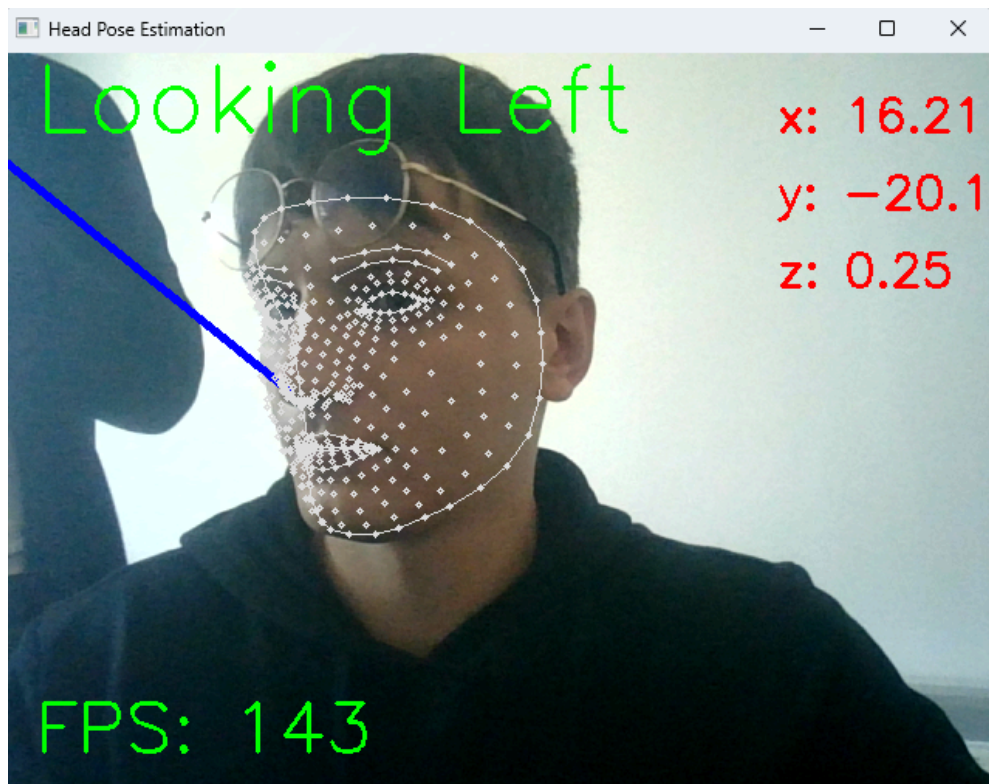
```



Obrazek 1. Spojrzenie w przód.



Obrazek 2. Spojrzenie w dół.



Obrazek 3. Spojrzenie w bok.